

MODULE 5 · STORAGE

Volumes

Giving containers storage that outlives a restart — or doesn't

A container's filesystem dies with the container

Every container starts fresh from its image layers. Anything it writes to its own filesystem is gone the moment that container restarts — a crash-looping app loses its own logs, and two containers in the same Pod have no way to hand data to each other.

A **volume** is a Pod-level storage source that containers mount into their own filesystem. How long it lasts, and where it lives, depends on the volume type.

BY THE END YOU CAN

- Share files between containers in a Pod
- Explain why emptyDir empties on Pod deletion
- Mount a node path into a container
- Say why hostPath doesn't belong in production

CONCEPT

emptyDir: a shared, Pod-lifetime scratch space

An emptyDir volume is created empty the moment the Pod is scheduled to a node. Every container in that Pod can mount it — at the same path or a different one — which is exactly how two containers share files with no external storage system involved.

Its lifetime is tied to the **Pod**, not any one container. A container can crash and restart and the data survives; delete or reschedule the Pod itself and the volume is deleted with it.



Same Volume API, opposite trade-offs

EMPTYDIR	HOSTPATH
Backing storage: a kubelet-managed directory	Backing storage: an exact path you choose on the node
Lifetime: created with the Pod, deleted with the Pod	Lifetime: outlives the Pod — persists on that node's disk
Portability: node-agnostic — Pod can land anywhere	Portability: tied to one specific node
Typical use: cache, scratch space, sidecar hand-off	Typical use: a node-level agent reading host logs/sockets

RULE OF THUMB

Reach for emptyDir by default. Only use hostPath when a Pod genuinely needs the node's own filesystem — and keep the path as narrow as possible.

Declared once, mounted by name — two containers, one emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
  namespace: training
spec:
  containers:
    - name: writer
      image: busybox:1.36
      command:
        - /bin/sh
        - -c
        - "while true; do echo \"$(date) - Hello from writer\" >> /data/log.txt; sleep 5; done"
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox:1.36
      command:
        - /bin/sh
        - -c
        - "tail -f /data/log.txt"
      volumeMounts:
        - name: shared-data
          mountPath: /data # same volume, same path
  volumes:
    - name: shared-data
      emptyDir: {} # created empty, Pod-scoped
```

volumes is Pod-level and declared once; **volumeMounts** is per-container — the matching `name:` is what links writer and reader to the same `shared-data` volume, here both mounted at `/data`.

Dry-run gives you the skeleton, not the volume

IMPERATIVE — SKELETON ONLY

```
kubectl run emptydir-pod --image=busybox:1.36
```

No `--volume` flag exists — you get a bare container

DECLARATIVE — FULL VOLUME SPEC

```
kubectl apply -f pod-emptydir.yaml
```

You hand-add `volumes:` and `volumeMounts:` yourself

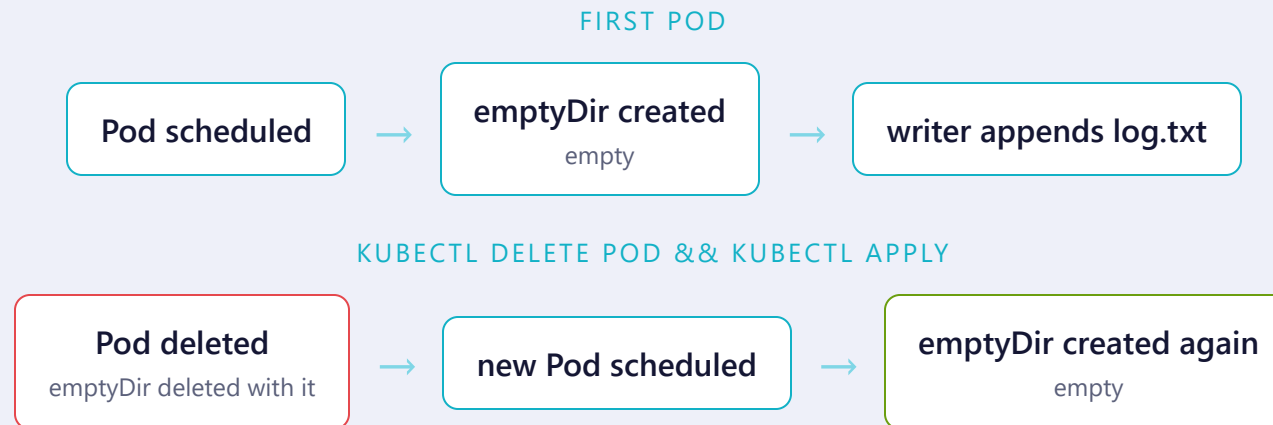
THE BRIDGE TRICK

Generate the container skeleton, then edit the volume blocks in by hand:

```
$ kubectl run emptydir-pod --image=busybox:1.36 \  
  --dry-run=client -o yaml > pod.yaml  
# then add volumes: + volumeMounts: before applying
```

SEE IT

Delete the Pod, and the data goes with it

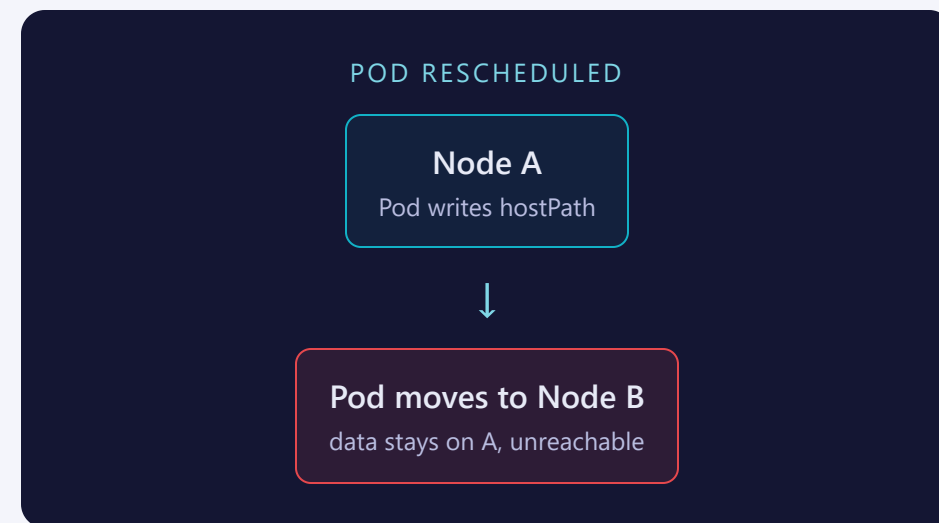


`kubectl exec -c writer -- cat /data/log.txt` after a delete+recreate shows only new entries — the old file isn't truncated, it never existed for this Pod.

hostPath ties your Pod to one node

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
  namespace: training
spec:
  containers:
    - name: writer
      image: busybox:1.36
      command:
        - /bin/sh
        - -c
        - "echo 'Written by hostPath pod' > /host-data/test.txt && sleep 3600"
      volumeMounts:
        - name: host-volume
          mountPath: /host-data
  volumes:
    - name: host-volume
      hostPath:
        path: /tmp/k8s-lab-data
        type: DirectoryOrCreate # create the dir if missing
```

A hostPath volume mounts a real directory from the node's own disk. That makes it feel more durable than emptyDir — the file really is still there after the Pod is deleted — but only on **that one node**.



No error, no warning — the old data just isn't reachable. The path is also whatever the node lets you read or write.

Where people trip

- **Expecting emptyDir to survive a Pod delete.** It's Pod-scoped, not container-scoped — deleting and recreating the Pod wipes it even though nothing else changed.
- **Missing host directory.** Without type: `DirectoryOrCreate`, a `hostPath` mount fails outright if the node path doesn't exist yet.
- **hostPath after a reschedule.** Pod lands on a different node → the file you wrote is nowhere to be found, with no error to tell you why.
- **Treating hostPath as shared storage.** It's local to one node's disk — not a substitute for real network-backed storage when multiple nodes need the same data.

Commands to keep at hand

```
$ kubectl exec POD -c CONTAINER -- cat /path    # read from one container
$ kubectl describe pod POD                      # volumes + mounts section
$ kubectl get pod POD -o jsonpath='{.spec.nodeName}' # which node it's on
$ kubectl delete pod POD --force --grace-period=0 # test a volume's lifetime
$ kubectl apply -f pod.yaml                     # recreate, watch data reset
```

Tip: `-c CONTAINER` is required the moment a Pod has more than one container — both `logs` and `exec` need it.

RECAP

Volumes in one breath

- `emptyDir`: created with the Pod, shared by all its containers, gone when the Pod is
- `hostPath`: a literal node path — persists per-node, but isn't portable
- `volumes`: declared once at Pod level; `volumeMounts`: wires it into each container
- Default to `emptyDir`; reach for `hostPath` only for genuine node-level needs

HANDS-ON

Now run Lab 5.1
[labs/5.1/](#)

Next: [5.2 — PersistentVolumes & PVCs](#)